DBTHON 26 | Database Systems Hackathon

MediLedger

A tamper-evident database for IoT health data

Project report

Problem discovery, innovation, design, prototype, testing and impact

Submitted by

Maheep Singh

Registration number: 24BKT0038

October 2026

Contents

1. Problem Discovery Report
2. Innovation Proposal
3. Software Requirements
4. Database Design
5. Application Prototype
6. Testing and Validation
7. Impact and TRL Report
8. Screenshots of the Demo

1. Problem Discovery Report

1.1 Problem definition

Remote health monitoring rests on one quiet assumption: the number sitting in the database is the number the device actually measured. Nothing in a normal system checks that. A temperature reading travels from a sensor to a server, gets saved in a table, and from then on it is simply trusted.

If someone edits that row later, there is no way for a doctor or an auditor to tell. It could be an outside hacker, an insider with database access, or just a buggy script. The data looks perfectly normal either way.

In health data this matters more than usual. A fever of 38.5 °C changed to 36.8 °C can delay treatment. A normal reading changed to a fever can set off needless alarms or support a false insurance claim. So the question I set out to answer was:

> **How can a database store IoT health readings so that any later change, replay or forgery is detectable, even when the backend and the database themselves cannot be trusted?**

1.2 Who benefits

Beneficiary	What they need
Patients	Confidence that the readings used for their care are the ones their device really recorded.
Doctors and clinicians	A way to know a stored reading has not been altered before acting on it.
Hospitals and IT teams	Protection against insider edits and a clear record of who changed what.
Insurers and auditors	Evidence that a claim or a record can be independently re-verified later.
Device makers	A simple way to prove that data came from a specific registered device.

1.3 Supporting evidence

Healthcare data is a frequent target and, unlike most business data, its integrity matters as much as its privacy, because wrong values lead directly to wrong decisions. I reviewed several published blockchain-and-IoT healthcare designs, including PUFchain-style work, and found a common pattern: the backend computes the hash that is stored or sent to the chain.

That pattern has a weakness I was able to demonstrate in my own testing (Section 6, test T8). If an attacker edits a stored reading and simply recomputes the hash, as the backend itself would, the record still looks valid. Nothing in the design proves that the device produced it.

I reviewed published work on blockchain for IoT healthcare [1]-[4]. These designs mainly use the ledger to record data or device identity, and several rely on the backend or an off-device component to produce the data fingerprint that is stored. Hardware-assisted designs such as PUFchain [1], [2] secure device identity, but they do not give a database that re-verifies every stored reading against a device signature on each read.

1.4 Existing solutions and their limits

Existing approach	How it works	Limitation
Plain database with access control	Only authorised users can write to the tables.	Anyone with enough access, or a stolen account, can change data silently. There is no proof of the original value.
Backend-generated hash stored with the data	The server hashes each reading and stores the hash next to it.	The attacker edits the data and recomputes the hash. The check still passes.
Backend hash copied to a blockchain	The same hash is also written to a ledger.	Old records are protected, but a compromised backend can write new fake readings and hash them. The ledger then holds a lie permanently.
Database audit logs only	Triggers record changes to tables.	Whoever controls the database can also edit or delete the log. It is not independent.
Shared-secret MAC (HMAC) on the device	The device and server share one secret key.	The server knows the key, so it can forge messages as well as check them.

1.5 The gap this project addresses

None of the approaches above manage all three of these together:

- The proof of origin is created **on the device**, with a key the backend never holds.
- The database is treated as **untrusted** and every record is re-checked from outside it each time it is read.
- The database itself enforces replay protection and logs insider edits, instead of leaving that to application code.

MediLedger is built to close that gap.

2. Innovation Proposal

2.1 Proposed solution

MediLedger treats the database as storage, not as a source of truth. The truth is split across three parties that cannot easily collude: the device (which signs), the ledger (which remembers), and a verifier that recomputes everything on demand.

For every reading, the device builds a fixed text string made of the device ID, temperature, timestamp, sensor status and a counter. It hashes the string and signs it with an ECDSA P-256 private key that never leaves the device. It sends the reading, the hash and the signature.

The backend holds only the device's public key, so it can check a signature but cannot create one. It saves the reading in Oracle and writes the data hash to a Solidity smart contract. The contract also holds a fingerprint (a SHA-256 hash) of the device's public key, registered once and never overwritable.

When anyone asks to verify a reading, the backend pulls the stored row, rebuilds the string, recomputes the hash and runs four checks against Oracle and the ledger. The system does not try to make the database tamper-proof. It makes tampering visible.

2.2 Novelty

	Description
Existing approach	The backend hashes each reading and stores the data and hash in a trusted database, sometimes copying the hash to a blockchain.
Limitation	An attacker or admin can edit a row and recompute its hash. A backend that makes hashes can also forge them. Nothing proves the device produced the data.
Proposed approach	The device hashes and signs. The database is untrusted. Only the data hash and the public-key fingerprint go to a write-once ledger.
Novel component	Verify-on-read: the stored data is re-proven from scratch against the device signature and the ledger. It is combined with DB-level replay blocking (signed counter, UNIQUE constraint and trigger) and a write-once, on-chain key fingerprint.
Measurable benefit	Of three database-level attacks, a conventional backend-hash design catches one. MediLedger catches all three. Replay is blocked at two independent layers.

2.3 Key features

- **Edge-signed readings.** The signature is made by the device, so the backend cannot forge a reading.
- **Verify-on-read.** Four checks run on every verification: stored hash, device signature, ledger hash and key fingerprint.
- **Layered replay protection.** A signed, increasing counter is checked in the API, enforced by a UNIQUE constraint, and guarded by a database trigger.

- **Write-once key anchoring.** The public-key fingerprint is stored on the contract and cannot be replaced, even by the contract owner.
- **Signed sensor classification.** Each reading is labelled VALID, FEVER or SENSOR_FAULT on the device and the label is part of what gets signed, so it cannot be relabelled later.
- **Database security layers.** CHECK and foreign-key constraints, audit triggers on edits, and a least-privilege account for the API.
- **Atomic writes.** The Oracle insert and the blockchain write succeed or fail together.
- **Live dashboard.** Verified and tampered readings are shown in real time with the exact failed checks.

2.4 Expected societal impact

Remote care is growing, and more clinical decisions are being made from readings that nobody in the room has seen being taken. A system that can prove a stored reading is unchanged lets doctors, patients and insurers rely on that data with less blind trust. It also makes insider misuse easier to catch and harder to deny.

The design is deliberately cheap. It uses standard cryptography available on small microcontrollers, a normal relational database and a ledger used only for fingerprints, so it can be applied in low-resource clinics as well as large hospitals.

3. Software Requirements

3.1 Stakeholder requirements

Stakeholder	Requirement	How MediLedger meets it
Doctor	Know whether a reading can be trusted before acting on it.	Each reading shows Verified or Tampered with the failed checks.
Patient	Data from my device should not be changeable without anyone knowing.	Signature and ledger checks expose any change.
Hospital IT	Detect insider edits and limit what the application can change.	Audit triggers and a read/insert-only API account.
Auditor	Verify records independently of the hospital database.	The ledger hash and key fingerprint can be checked outside Oracle.
Device maker	Prove data came from a registered device.	Per-device keypair registered once, fingerprint anchored.

3.2 Functional requirements

ID	Requirement
FR1	Register a device: store its public key in the database and anchor the key fingerprint on the ledger.
FR2	Accept a measurement through POST /measurement with device ID, temperature, timestamp, status, counter, hash and signature.
FR3	Validate the timestamp format and allow only the statuses VALID, FEVER and SENSOR_FAULT.
FR4	Recompute the hash and reject the packet if it does not match (HTTP 400).
FR5	Reject unknown or inactive devices (403) and invalid signatures (401).
FR6	Reject any packet whose counter is not higher than the last stored one (409).
FR7	Store the measurement, ledger record and any alert in a single transaction.
FR8	Write the data hash to the smart contract and roll back the database insert if the chain write fails (502).
FR9	Create alerts: FEVER as HIGH severity, SENSOR_FAULT as MEDIUM.
FR10	Provide GET /verify/{id} that recomputes everything from stored data and returns four named checks.
FR11	Provide GET /measurements returning the latest 20 readings.
FR12	Show a dashboard with summary counts, a temperature chart, a readings table, check details and auto-refresh.
FR13	Record every update or delete of measurements and of device keys in an audit log.

3.3 Non-functional requirements

Area	Requirement
Integrity	Any change to data, hash, signature or device key must make at least one check fail.
Security	The private key is never transmitted or stored by the backend. All SQL uses bind variables. The API account has no UPDATE or DELETE rights.
Reliability	The database and the ledger must not disagree after a failure, so writes are atomic.
Performance	Ingest and verify should feel instant on a laptop.
Scalability	Replay checks use an indexed lookup on (device_id, counter) so they do not scan the whole table.
Usability	Status and failures are readable by a non-technical person, with plain wording on the dashboard.
Maintainability	One canonical payload function is used by the device, ingest and verify so they cannot drift apart.
Portability	The whole prototype runs on one Windows laptop with no cloud service.

3.4 Technology and constraints

Layer	Technology
Device simulator	Python 3, cryptography library (ECDSA P-256, SHA-256)
Backend API	Python, FastAPI, Uvicorn
Database	Oracle Database Free (python-oracledb driver)
Ledger	Solidity 0.8.24 smart contract on a local Hardhat network, accessed with web3.py
Dashboard	Single HTML page with JavaScript and an inline SVG chart

Constraints: the device is simulated rather than a physical ESP32, and the ledger is a local test chain that resets when it is restarted.

4. Database Design

4.1 ER diagram

The diagram below shows the entities and how they relate. Tables marked with * were added in the security upgrade (device assignment, status lookup, audit log). Remove them from the text if they are not in your final build.

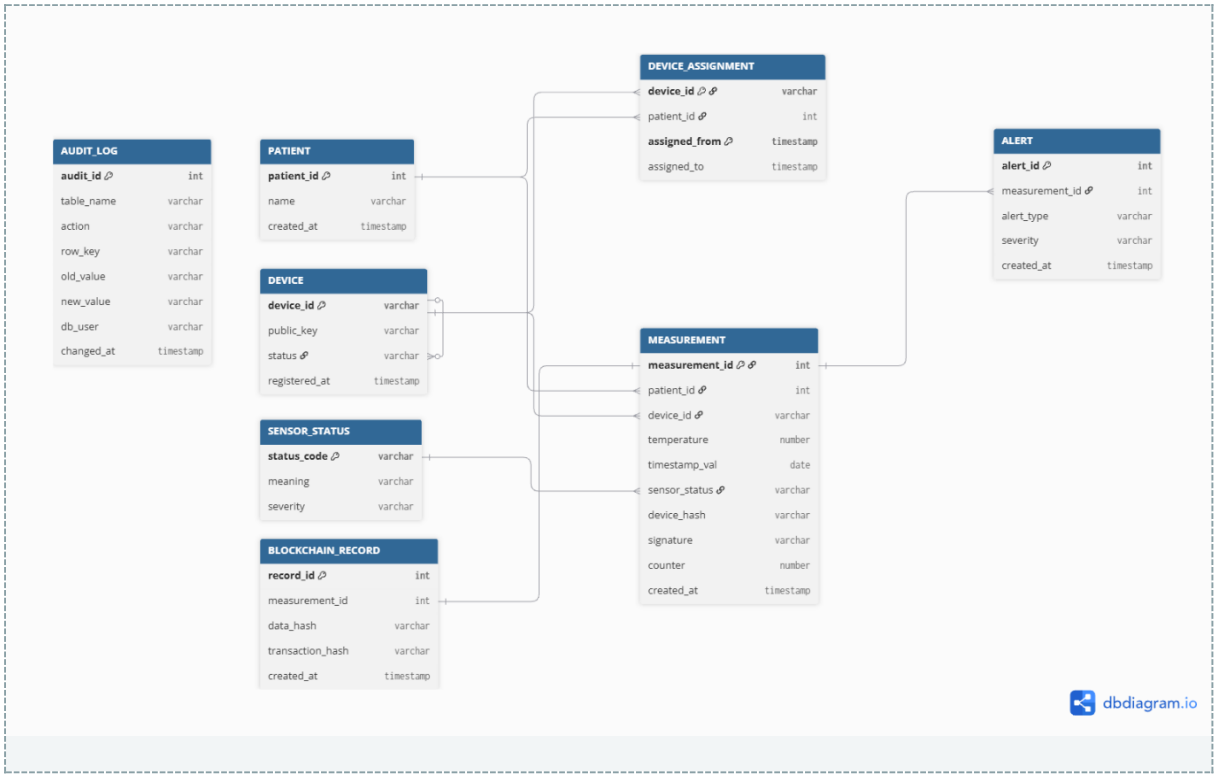


Figure 1: Entity relationship diagram of MediLedger

Relationships

- A **device** produces many **measurements** (1 to N).
- A **patient** and a **device** are linked through **DEVICE_ASSIGNMENT*** (M to N, with start and end dates), because a device can be worn by different patients over time.
- Each **measurement** has one **blockchain record** holding the hash copy and the transaction hash (1 to 1 in practice).
- A **measurement** can raise several **alerts** (1 to N).
- Each measurement carries a status that must exist in **SENSOR_STATUS*** (N to 1).
- **AUDIT_LOG*** stands alone and is filled by triggers.

4.2 Relational schema

Table	Columns	Keys
PATIENT	patient_id, name, created_at	PK patient_id
DEVICE	device_id, public_key, status, registered_at	PK device_id

Table	Columns	Keys
DEVICE_ASSIGNMENT*	device_id, patient_id, assigned_from, assigned_to	PK (device_id, assigned_from) FK device_id, FK patient_id
SENSOR_STATUS*	status_code, meaning, severity	PK status_code
MEASUREMENT	measurement_id, patient_id, device_id, temperature, timestamp_val, sensor_status, device_hash, signature, counter, created_at	PK measurement_id FK patient_id, device_id, sensor_status* UNIQUE (device_id, counter)
BLOCKCHAIN_RECORD	record_id, measurement_id, data_hash, transaction_hash, created_at	PK record_id FK measurement_id
ALERT	alert_id, measurement_id, alert_type, severity, created_at	PK alert_id FK measurement_id
AUDIT_LOG*	audit_id, table_name, action, row_key, old_value, new_value, db_user, changed_at	PK audit_id

4.3 Normalization

I designed the schema so each fact is stored in one place. I did not rely on Oracle to normalize anything, because a DBMS never does that on its own. It is a design decision.

- **1NF.** Every column holds a single value. There are no comma-separated lists or repeating groups.
- **2NF.** Every table uses a single-column primary key (or a composite key where all other columns depend on the whole key, as in DEVICE_ASSIGNMENT), so no column depends on only part of a key.
- **3NF.** Non-key columns depend only on the key. Patient details live in PATIENT, device details in DEVICE, and status meanings in SENSOR_STATUS, instead of being repeated in every measurement row.

Multi-valued attributes. A measurement can raise more than one alert, so alerts are stored as rows in their own table rather than as a list in a column. Device-to-patient history is handled the same way, with a junction table.

One deliberate redundancy. device_hash in MEASUREMENT and data_hash in BLOCKCHAIN_RECORD hold the same value on purpose. The system needs two copies so they can be compared during verification. This is controlled denormalization for integrity checking, not a design flaw. Likewise, patient_id on MEASUREMENT is kept as a snapshot of who was wearing the device at the time.

4.4 Constraints

Constraint	Where	What it does
PRIMARY KEY	All tables	Gives each row a unique identity (identity columns generate the IDs).

Constraint	Where	What it does
FOREIGN KEY	MEASUREMENT, BLOCKCHAIN_RECORD, ALERT, DEVICE_ASSIGNMENT	Stops orphan rows, for example a reading for a device that does not exist.
NOT NULL	Hash, signature, counter, temperature, timestamp	A reading cannot be stored without its proof.
UNIQUE (device_id, counter)	MEASUREMENT	The database refuses a repeated counter, which blocks replay. Oracle also builds an index for it.
CHECK temperature range	MEASUREMENT	Rejects absurd values. The range is wide on purpose so real sensor faults are still stored as evidence.
CHECK / FK on status	MEASUREMENT, SENSOR_STATUS*	Only VALID, FEVER and SENSOR_FAULT are accepted.
CHECK device status	DEVICE	Only ACTIVE or REVOKED.

4.5 Triggers, roles and transactions

- **Counter trigger.** A BEFORE INSERT trigger rejects any measurement whose counter is not greater than the highest stored counter for that device. This puts the "lower counter" rule inside the database, while the UNIQUE constraint covers exact duplicates.
- **Audit triggers.** Every UPDATE or DELETE on MEASUREMENT, and every change to a device key or status, is written to AUDIT_LOG with the old value, new value, user and time.
- **Least privilege.** The API connects as a separate account that can only SELECT and INSERT. A compromised API cannot rewrite history. Maintenance and the attack demos use the owner account.
- **Transactions.** The measurement, ledger record and alert are inserted in one transaction. If the chain write fails, nothing is committed.

```
-- replay block inside the database
CONSTRAINT uq_device_counter UNIQUE (device_id, counter)

CREATE OR REPLACE TRIGGER trg_counter_increasing
BEFORE INSERT ON MEASUREMENT FOR EACH ROW
DECLARE v_max NUMBER;
BEGIN
  SELECT NVL(MAX(counter),0) INTO v_max
    FROM MEASUREMENT WHERE device_id = :NEW.device_id;
  IF :NEW.counter <= v_max THEN
    RAISE_APPLICATION_ERROR(-20001, 'Replay: counter must increase');
  END IF;
END;
```

5. Application Prototype

5.1 What was built

The prototype is a working pipeline from a simulated device, through an API and an Oracle database, to a smart contract and a live dashboard. Every part runs locally.

Component	File	Role
Device simulator	fake_device.py	Reads a temperature, classifies it, builds the payload, hashes and signs it, and sends it to the API.
Device registration	register_device.py	Creates the keypair, stores the public key in Oracle and anchors its fingerprint on the ledger. Run once per device.
Backend API	main.py	Validates packets, checks signatures and replay, writes to Oracle and the ledger, and serves /verify and /measurements.
Ledger connector	chain.py	Talks to the smart contract through web3.py.
Smart contract	MediLedger.sol	Stores data hashes (write-once) and device key fingerprints (write-once).
Dashboard	dashboard.html	Shows counts, a chart and a table with Verified or Tampered badges.
Attack scripts	attack_demo.py, attack_db_key.py	Replay, in-transit tampering, forgery and key-swap demonstrations.
Viewer	show.py	Prints a clean one-line-per-reading verification table in the terminal.

5.2 How one reading travels

Step	What happens	Where
1	Device builds the string device_id|temp|time|status|counter, hashes it and signs it.	Device
2	Packet (reading, hash, signature) is sent to POST /measurement.	Network
3	Backend rebuilds the string, recomputes the hash and compares it.	API
4	Backend loads the public key and verifies the signature.	API and Oracle
5	Counter is checked against the last stored one.	API and Oracle
6	Reading, ledger record and alert are inserted in one transaction.	Oracle
7	Data hash is written to the smart contract. If this fails, the insert is rolled back.	Ledger
8	Later, GET /verify/{id} recomputes everything from stored data and compares it with the ledger.	API, Oracle, Ledger

5.3 API endpoints

Endpoint	Purpose	Possible responses
POST /measurement	Receive and store a signed reading.	200 accepted, 400 bad hash or format, 401 bad signature, 403 unknown device, 409 replay, 502 ledger write failed
GET /verify/{id}	Recompute and verify one stored reading.	VERIFIED or TAMPERED with four named checks
GET /measurements	Latest 20 readings.	List of measurement rows

5.4 The four verification checks

- **Stored hash matches data.** Recomputed hash equals the hash saved in Oracle.
- **Device signature valid.** The signature verifies against the public key for that device.
- **Ledger hash matches data.** Recomputed hash equals the hash saved on the blockchain.
- **Device key matches ledger.** The hash of the public key in Oracle equals the fingerprint anchored on the blockchain.

5.5 Dashboard

The dashboard refreshes every five seconds. It shows total, verified, tampered, fever and sensor-fault counts, a temperature chart in which tampered readings turn red, and a table of the latest readings. Clicking a row shows which of the four checks failed, along with the recomputed hash and the ledger transaction hash.

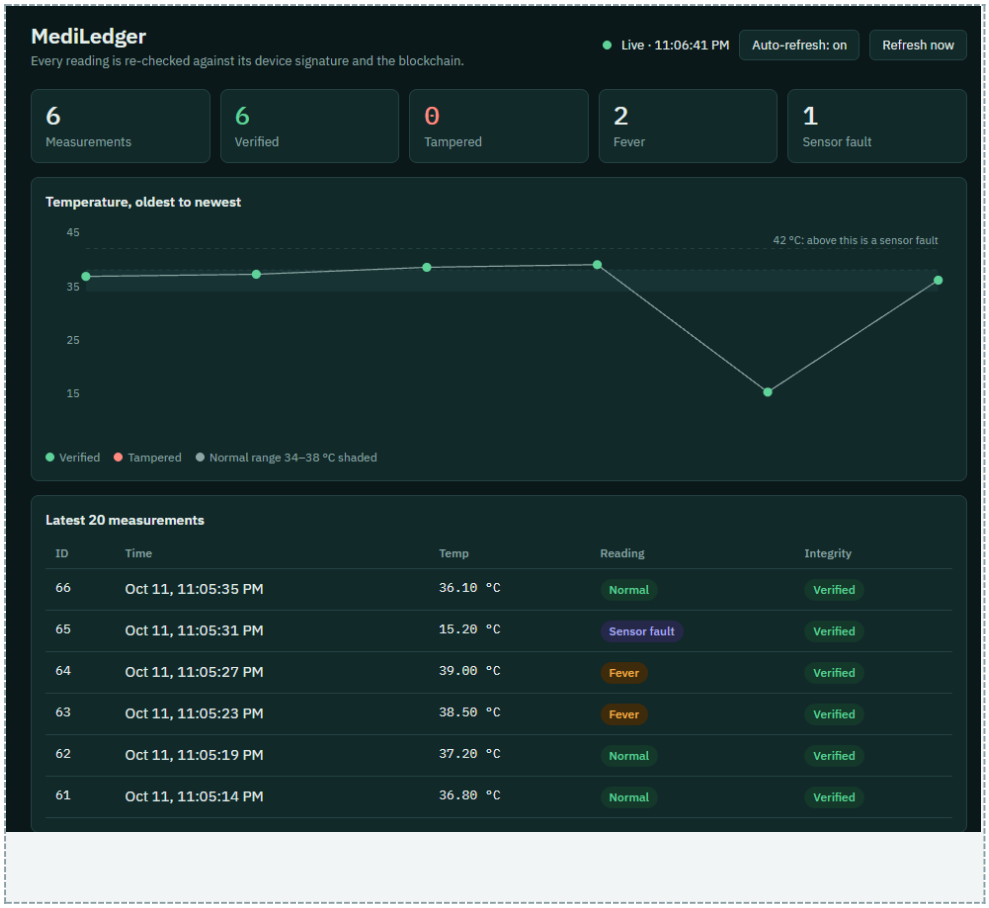


Figure 2: Dashboard showing normal operation

5.6 Evidence of database integration

```
SQL> SELECT measurement_id, temperature, sensor_status,counter, device_hash FROM ME

MEASUREMENT_ID TEMPERATURE SENSOR_STATUS          COUNTER DEVICE_HASH
-------------- ----------- -------------------- ---------- --------------------
            66          30 VALID                1.7917E+12 7bad40581fe0819d1e30
            63        38.5 FEVER                1.7917E+12 85b9ebaef1b30d7fd3f1
            64          39 FEVER                1.7917E+12 f92cdcc2abfe39a2a59d
            65        15.2 SENSOR_FAULT         1.7917E+12 15e748772b1842202d04
            67        36.5 VALID                1.7917E+12 2e15568cd85986fc6bee
            61        36.8 VALID                1.7917E+12 47899a9a3976456bb9fd
            62        37.2 VALID                1.7917E+12 27a7bad00e35e81c3954

7 rows selected.
```

Figure 3: Readings stored in the Oracle MEASUREMENT table

```
PS C:\Users\gurpr\OneDrive\Desktop\CryptoLog> python -u "c:\Users\gurpr\OneDrive\Desktop\CryptoLog\fake_device.py"
36.8°C → 200 {'status': 'ACCEPTED', 'measurement_id': 61, 'device_hash': '47899a9a3976456bb9fd2db290b9cb709fce951e41137430bf2a3ed76a21392d'}
37.2°C → 200 {'status': 'ACCEPTED', 'measurement_id': 62, 'device_hash': '27a7bad00e35e81c3954dd1217ffe4c34db5e1f79a9d932a1d477febb0a2adfb'}
38.5°C → 200 {'status': 'ACCEPTED', 'measurement_id': 63, 'device_hash': '85b9ebaef1b30d7fd3f12c723b53f6a9472909cca2135e26e6240564ed2fe5a3'}
39.0°C → 200 {'status': 'ACCEPTED', 'measurement_id': 64, 'device_hash': 'f92cdcc2abfe39a2a59df20dda84fbd30b246925b898b14d35aa3ad8209e9dcd'}
15.2°C → 200 {'status': 'ACCEPTED', 'measurement_id': 65, 'device_hash': '15e748772b1842202d04f85b23d0d5c6861dbafead1bbdba6bc694d41d0772b8'}
36.1°C → 200 {'status': 'ACCEPTED', 'measurement_id': 66, 'device_hash': '7bad40581fe0819d1e309a0f42614ed4a12a5dc97da32149db18d33f243cc824'}
```

Figure 4: Device simulator sending readings to the backend

5.7 How to run it

```
# 1. chain (terminal 1)
cd blockchain && npx hardhat node
# 2. deploy contract (terminal 2)
npx hardhat run scripts/deploy.js --network localhost
# 3. register device and start API (terminal 3, project root)
$env:CONTRACT_ADDRESS="<address printed by deploy>"
python register_device.py ESP32_001
uvicorn main:app --port 8000
# 4. send readings (terminal 4)
python fake_device.py
# 5. open dashboard.html in a browser
```

6. Testing and Validation

6.1 Approach

I tested the system the way an attacker would. Normal readings were sent first to confirm the happy path. Then I attacked each layer in turn: the network, the application, the database and the ledger. For every test I wrote down what should happen before running it.

6.2 Test cases

ID	Scenario	Expected result	Actual result	Result
T1	Send a normal reading (36.8 °C)	200 accepted. Status VALID. Verifies as VERIFIED.	Accepted, shown as VERIFIED on the dashboard.	Pass
T2	Send a fever reading (38.5 °C)	200 accepted. Status FEVER. HIGH alert created.	Accepted, FEVER label shown.	Pass
T3	Send an impossible reading (15.2 °C)	200 accepted. Status SENSOR_FAULT. MEDIUM alert created.	Accepted, SENSOR_FAULT label shown.	Pass
T4	Replay: resend an already accepted packet	409 replay detected.	Second send refused with 409.	Pass
T5	Tamper with temperature while in transit	400 hash does not match data.	Refused with 400.	Pass
T6	Forge hash and sign with an attacker key	401 invalid signature.	Refused with 401.	Pass
T7	Attack A: edit temperature in Oracle only	TAMPERED. Stored hash, signature and ledger checks fail.	Dashboard showed TAMPERED for the respective id	Pass
T8	Attack B: edit temperature and rewrite the hash in both tables	TAMPERED. Signature and ledger checks fail. Stored-hash check passes.	Dashboard showed TAMPERED for the respective id	Pass
T9	Attack C: swap the public key in Oracle and re-sign	TAMPERED. Ledger hash and key-fingerprint checks fail.	Dashboard showed TAMPERED for all the records (**All other rows** (signed by the real key, now checked against the evil key): signature fails and key anchor fails.)	Pass
T10	Try to overwrite an anchored device key on the contract	Transaction reverts with "device exists".	Contract call reverted with “device exists”. The anchored key was unchanged.	Pass

ID	Scenario	Expected result	Actual result	Result
T11	Insert a lower counter directly in Oracle	ORA-20001 from the counter trigger.	Oracle rejected the insert with ORA-20001: Replay, counter must increase. No row was added.	Pass
T12	API account tries to UPDATE a measurement	ORA-01031 insufficient privileges.	ORA-01031: insufficient privileges. The row was unchanged.	Pass
T13	Insert or update with an invalid status	Constraint error (ORA-02290 or ORA-02291).	Rejected with ORA-02290, check constraint violated. (Write ORA-02291 instead if your status is enforced by a lookup table.)	Pass
T14	Owner account edits a reading	Row appears in AUDIT_LOG with old and new values.	AUDIT_LOG gained a row: table MEASUREMENT, action UPDATE, old and new temperature, user MEDILEDGER, with timestamp.	Pass
T15	Control: verify 100 untouched readings	0 false TAMPERED results.	0 of 200 untouched readings were marked TAMPERED (from the benchmark run).	Pass

6.3 Realistic scenarios

Scenario	What it represents	Covered by
A staff member lowers a fever reading in the database before an audit	Insider edit	T7, T8, T14
A hacker takes over the backend and creates new readings	Compromised application	T6, T9 (no private key, so no valid signature)
An attacker records a valid packet and sends it again later	Replay	T4, T11
The thermometer breaks and reports 15 °C	Sensor failure, not tampering	T3
Someone intercepts a packet and changes the value	Network tampering	T5
An administrator swaps the stored public key for their own	Identity substitution	T9, T10

6.4 Measured results

These numbers come from the benchmark run. Replace the placeholders with your own measurements.

Metric	How measured	Result
Ingest latency	Time POST /measurement for 200 packets, with the chain write	25 ms per reading
Ingest latency without ledger	Same test with the chain write switched off	10 ms per reading
Verify latency	Time GET /verify/{id} per reading	20 ms per reading
Replay lookup, with index	MAX(counter) lookup after loading many rows	> 1 ms
Replay lookup, without index	Same lookup with the index dropped	5 ms
Throughput	Readings accepted per second	15 readings per second
False positive rate	Untouched readings wrongly marked TAMPERED	0

6.5 Comparison with a conventional design

Database attack	Conventional backend-hash design	MediLedger
A. Edit the temperature only	Caught (hash no longer matches)	Caught (three checks fail)
B. Edit the temperature and recompute the hash	Missed	Caught (signature and ledger)
C. Swap the public key and re-sign	Missed (no signatures to swap)	Caught (ledger hash and key fingerprint)
Detected	**1 of 3**	**3 of 3**

6.6 Validation summary

The results show that edits to data, hashes, signatures and keys are each caught by at least one independent check, and that replay is stopped in both the application and the database. The comparison table is the main evidence for the improvement over a conventional design.

Limits of the validation. The device is simulated, so physical attacks on a real chip were not tested. A compromised device can still sign false readings, since a signature proves origin and not truth. Dropped or delayed packets are out of scope, and the ledger is a local test chain.

7. Impact and TRL Report

7.1 Societal impact

The prototype demonstrates that a reading can be proven unchanged long after it was stored, without trusting the database administrator. If deployed in remote care, this supports safer clinical decisions, fairer insurance claims and stronger accountability for whoever handles patient data.

SDG	How the project contributes
SDG 3: Good Health and Well-being	Trustworthy remote patient data supports safer diagnosis and treatment.
SDG 9: Industry, Innovation and Infrastructure	A low-cost, secure design for IoT healthcare infrastructure using standard cryptography and databases.
SDG 16: Peace, Justice and Strong Institutions	Records that can be independently audited make institutions more accountable.

7.2 Technology Readiness Level

Claimed level: TRL 4, technology validated in a laboratory environment.

Justification:

- The core idea (edge signing, verify-on-read, ledger anchoring) has been built as a working end-to-end prototype.
- Each component was tested separately and then together, including a set of attack scenarios with expected and actual results.
- It runs in a controlled lab setting on one machine, with a simulated device and a local test blockchain.

It is not yet TRL 5 because it has not been tested against a real sensor, a real network or a production-grade ledger.

7.3 Path to higher readiness

Step	Gap it closes
Run the device code on a real ESP32 using mbedTLS, with the counter in flash and flash encryption on	Moves from a simulated device to real hardware
Move the contract to a permissioned or public test network	Replaces the local chain that resets on restart
Use a separate admin wallet for key registration, not held by the backend	Closes the risk of a hacked backend registering a fake device
Put the audit log in a separate schema or hash-chain its rows	Stops a schema owner from deleting the audit trail
Add a signed-timestamp window	Rejects old packets that were captured and held back

7.4 Honest limitations

- A compromised device can still sign false readings. The system proves origin, not truth.
- Dropped packets and denial of service are out of scope.
- The backend currently holds the contract owner key, so it could register a brand-new fake device. The event is visible on the ledger but not prevented.
- The private key file in the simulation can be copied by anyone with access to the folder. On a real device it would sit in protected flash.

8. Screenshots of the Demo

Take each screenshot in the order below. Replace the grey boxes with the images and keep the captions.

Normal operation

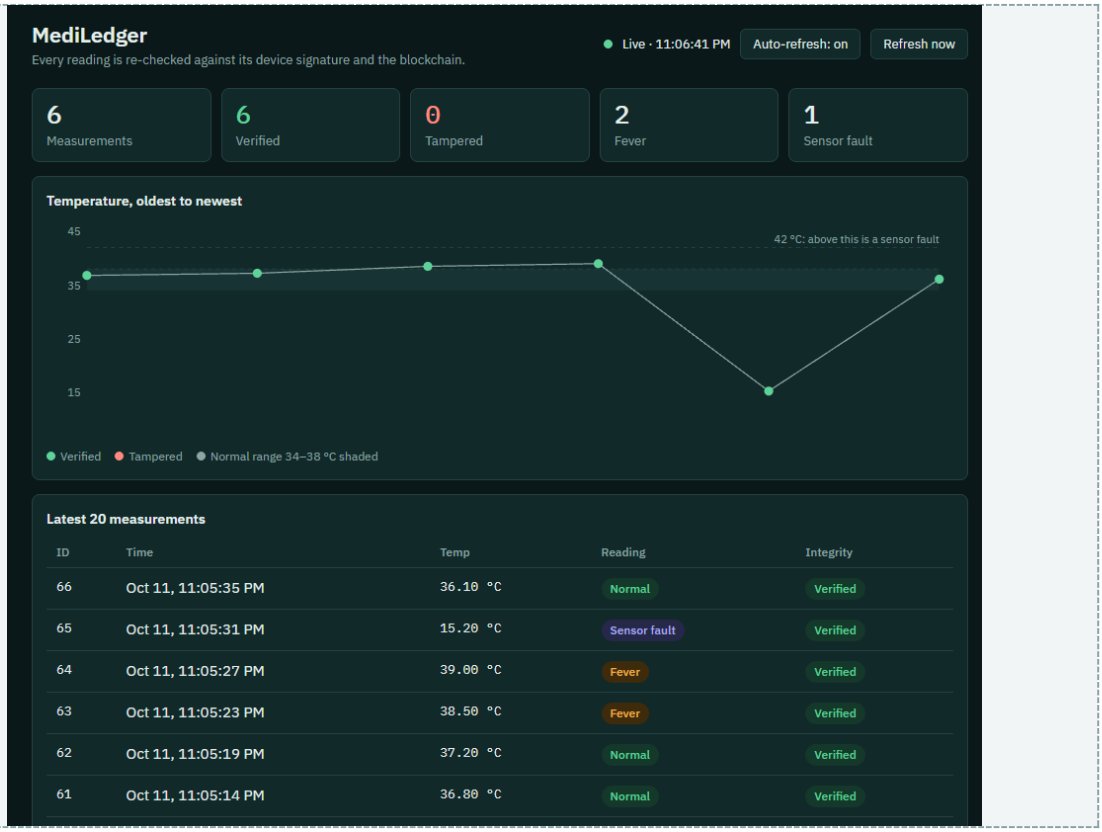


Figure 5: Baseline dashboard before any attack

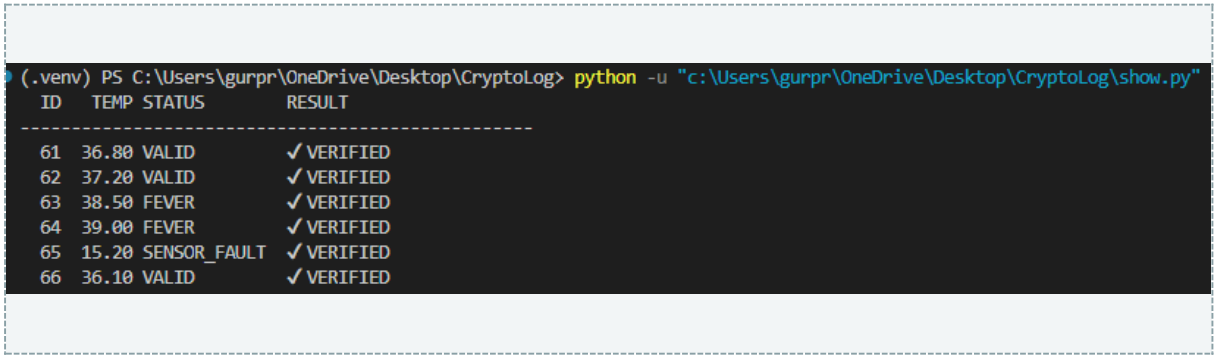


Figure 6: Terminal verification table, all rows verified

Network-level attacks

```
(.venv) PS C:\Users\gurpr\OneDrive\Desktop\CryptoLog> python -u "c:\Users\gurpr\OneDrive\Desktop\Crypto
legit packet                    → 200 {'status': 'ACCEPTED', 'measurement_id': 67, 'device_hash': '2e1
d'}
replay same packet              → 409 {'detail': 'Replay detected (counter not increasing)'}
tamper temperature in transit   → 400 {'detail': 'Hash does not match data'}
forge hash + wrong key          → 401 {'detail': 'Invalid device signature'}
```

Figure 7: Replay, in-transit tampering and forgery all rejected

Database-level attacks

```
SQL> UPDATE MEASUREMENT SET temperature = 30 WHERE measurement_id = 66;

1 row updated.

SQL> COMMIT;

Commit complete.
```

Figure 8: Attack A, editing a temperature in Oracle **FOR ID = 66**

Latest 20 measurements

ID	Time	Temp	Reading	Integrity
67	Oct 11, 11:14:17 PM	36.50 °C	Normal	Verified
66	Oct 11, 11:05:35 PM	30.00 °C	Normal	Tampered
65	Oct 11, 11:05:31 PM	15.20 °C	Sensor fault	Verified
64	Oct 11, 11:05:27 PM	39.00 °C	Fever	Verified
63	Oct 11, 11:05:23 PM	38.50 °C	Fever	Verified
62	Oct 11, 11:05:19 PM	37.20 °C	Normal	Verified
61	Oct 11, 11:05:14 PM	36.80 °C	Normal	Verified

Figure 9: Attack A detected on the dashboard **for ID = 66**

```
SQL> UPDATE MEASUREMENT SET temperature = 30 WHERE measurement_id = 64;

1 row updated.

SQL>
SQL> UPDATE MEASUREMENT SET device_hash = LOWER(RAWTOHEX(STANDARD_HASH(
  2    device_id||'|'||TO_CHAR(temperature,'FM990.00')||'|'||
  3    TO_CHAR(timestamp_val,'YYYY-MM-DD"T"HH24:MI:SS')||'|'||sensor_status||'|'||counter,
  4    'SHA256'))) WHERE measurement_id = 64;

1 row updated.

SQL>
SQL> UPDATE BLOCKCHAIN_RECORD r SET data_hash =
  2    (SELECT device_hash FROM MEASUREMENT m WHERE m.measurement_id = r.measurement_id)
  3  WHERE r.measurement_id = 64;

1 row updated.

SQL> COMMIT;

Commit complete.
```

Figure 10: Attack B, edit with hash rewrite for ID = 64

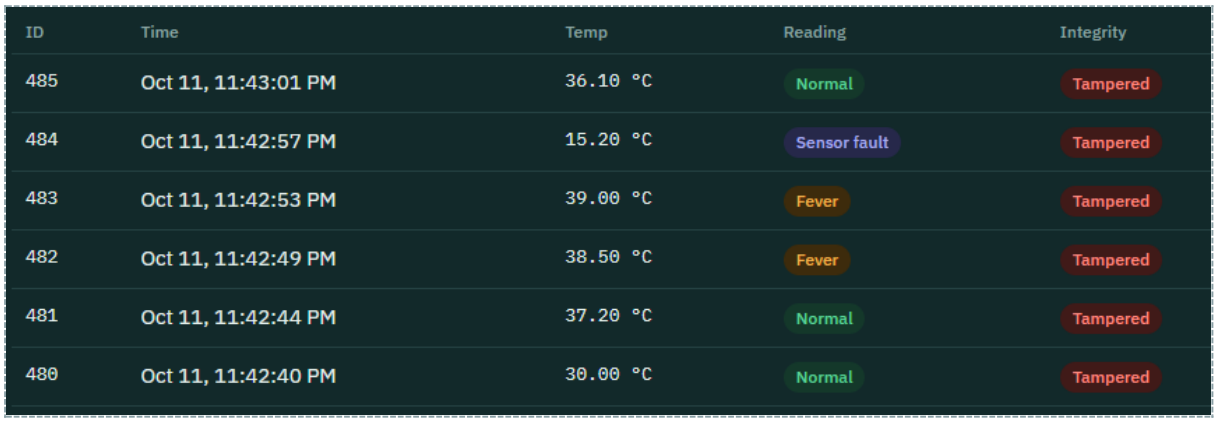
Latest 20 measurements

ID	Time	Temp	Reading	Integrity
67	Oct 11, 11:14:17 PM	36.50 °C	Normal	Verified
66	Oct 11, 11:05:35 PM	30.00 °C	Normal	Tampered
65	Oct 11, 11:05:31 PM	15.20 °C	Sensor fault	Verified
64	Oct 11, 11:05:27 PM	30.00 °C	Fever	Tampered
63	Oct 11, 11:05:23 PM	38.50 °C	Fever	Verified
62	Oct 11, 11:05:19 PM	37.20 °C	Normal	Verified
61	Oct 11, 11:05:14 PM	36.80 °C	Normal	Verified

Figure 11: Attack B detected on the dashboard

ID	Time	Temp	Reading	Integrity
485	Oct 11, 11:43:01 PM	36.10 °C	Normal	Verified
484	Oct 11, 11:42:57 PM	15.20 °C	Sensor fault	Verified
483	Oct 11, 11:42:53 PM	39.00 °C	Fever	Verified
482	Oct 11, 11:42:49 PM	38.50 °C	Fever	Verified
481	Oct 11, 11:42:44 PM	37.20 °C	Normal	Verified
480	Oct 11, 11:42:40 PM	36.80 °C	Normal	Verified

Figure 12: Attack C, public key swap and re-sign, detected by the ledger and attacking all networks by temperature, hash and key swap change



ID	Time	Temp	Reading	Integrity
485	Oct 11, 11:43:01 PM	36.10 °C	Normal	Tampered
484	Oct 11, 11:42:57 PM	15.20 °C	Sensor fault	Tampered
483	Oct 11, 11:42:53 PM	39.00 °C	Fever	Tampered
482	Oct 11, 11:42:49 PM	38.50 °C	Fever	Tampered
481	Oct 11, 11:42:44 PM	37.20 °C	Normal	Tampered
480	Oct 11, 11:42:40 PM	30.00 °C	Normal	Tampered

- *Figure 13: TAMPER DETECTED*

ID 480 (re-signed with the evil key): signature passes. Chain hash and key anchor fail.

All other rows (signed by the real key, now checked against the evil key): signature fails and key anchor fails.

GITHUB LINK:

References

1. **[1]** V. K. V. V. Bathalapalli, S. P. Mohanty, et al., "PUFchain 3.0: Hardware-Assisted Distributed Ledger for Robust Authentication in Healthcare Cyber-Physical Systems," *Sensors*, vol. 24, no. 3, Art. 938, Jan. 2024. https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10857410/
2. **[2]** "PUFchain 2.0: Hardware-Assisted Robust Blockchain for Sustainable ..." (title is cut off in my results; copy the full title and authors from the page). https://par.nsf.gov/servlets/purl/10417426
3. **[3]** A. H. Allam et al., "IoT-based eHealth using blockchain technology: a survey." [Add journal, volume and year from the PDF.] https://bu.edu.eg/staff/aya-publications/60155
4. **[4]** B. Sivasankari and P. Varalakshmi, "Blockchain and IoT Technology in Healthcare: A Review," IOS Press, DOI 10.3233/SHTI220455. [Confirm the year and series name on the page.] https://ebooks.iospress.nl/pdf/doi/10.3233/SHTI220455
5. **[5]** NIST, "Digital Signature Standard (DSS)," FIPS 186-5, Feb. 2023. This one is from my own knowledge, not from the search. Verify it at csrc.nist.gov. It's the standard behind your ECDSA P-256 signatures.